

ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA

2º Ingeniería Matemática e Inteligencia Artificial



Memoria del proyecto – Fundamentos de la Inteligencia Artificial

Autor: Javier Arraiza Arribas (202407851)

12/2025

Tabla de contenidos

Introducción	3
Planteamiento general de la estructura	3
PARTE 1	4
Acciones del juego	4
Juego interactivo	5
Juego automático con BFS	5
Main	6
PARTE 2	6
Entorno y generación	6
Bayes: Priors, Riesgo, Decisión	7
Acciones del juego	8
Juego interactivo	8
Juego automático Bayes	8
Main	9
Proceso de Decisión de Markov (MDP)	10
Entorno del río	10
Value Iteration	10
Simulación del río	11
Kurtz.py	11
Conclusión	11

Introducción

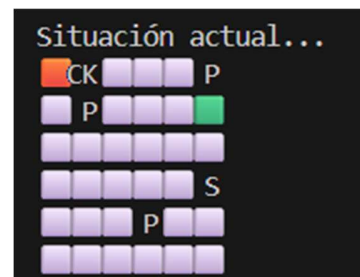
En este documento se presenta la memoria completa del proyecto que sigue el enunciado oficial. La estructura sigue la siguiente organización: Planteamiento general de la estructura, y código para la parte guiada, para la parte automática (con BFS) y para la parte del agente lógico, y por último, explicación del Proceso de Decisión de Markov (MDP).

Planteamiento general de la estructura

La parte de códigos que rigen las decisiones se basan en las funciones que crean, actúan, imprimen y comprueban.

La capa más baja de este problema la he planteado a través de un diccionario (dicc_entorno), y una clase (Celda). Este diccionario contiene toda la información necesaria para imprimir, moverse, o interactuar con el juego. Las claves constan de tuplas (x,y), donde cada una de ellas representan una casilla del tablero siguiendo la notación estándar. Debido a como empecé a implementar el diccionario, las casillas se imprimen empezando por la última fila, como si hubiésemos cogido de referencia el cuarto cuadrante.

Por ejemplo, la casilla de inicio, que por definición predeterminada es (1,1), se imprime en la esquina superior izquierda.



```
def __init__(self):
    self.celda = " "
    self.brisa = False
    self.ronquido = False
    self.resplandor = False
    self.pared_N = False
    self.pared_S = False
    self.pared_E = False
    self.pared_O = False
    self.grito = False
    self.es_salida = False
    self.tengo_CK = False
```

Los valores de estas claves constan de objetos Celda(), con diferentes atributos que afectarán al juego:

Atributos:

- celda (str): Contenido visual de la celda ("■", "■", "■", "S", "P", "CK").
- brisa (bool): Indica si hay brisa (cerca de precipicio).

- ronquido (bool): Indica si se oye ronquido (cerca de soldado).
- resplandor (bool): Indica si hay luz (cerca de salida).
- pared_N/S/E/O (bool): Indican si hay una pared en una de las direcciones.
- grito (bool): Indica si la celda del soldado donde se arroja la granada es transitable (es decir, "matamos" a este soldado).
- es_salida (bool): True si la celda es la salida.
- tengo_CK (bool): True si el capitán lleva al coronel con él.

He usado este diseño porque permite mantener toda la información relevante de cada casilla, de modo que el juego solo tiene que consultar atributos de la celda actual y de la celda destino.

La generación del palacio crea todas las celdas vacías, coloca al capitán en la entrada ■, que de forma predeterminada, es (1,1), coloca una salida ■ en una casilla aleatoria distinta de la entrada (marca esta casilla con es_salida), coloca peligros como precipicios o el soldado, así como al Coronel Kurtz en casillas aleatorias libres. Una vez colocados los elementos, ejecutamos la función comprobaciones que marcan las percepciones a las celdas adyacentes. Por ejemplo, una casilla cerca de un precipicio tendrá brisa, una casilla cerca del soldado tendrá ronquido y una casilla cerca de la salida tendrá resplandor.

Para automatizar la visualización, creé la función imprimir_tablero, que transforma en una representación tipo matriz que se imprime por consola. La conversión concatenando el símbolo de cada celda en filas, añadiendo cabeceras de coordenadas para facilitar la lectura.

PARTE 1

Acciones del juego

Estas acciones son el "motor" del juego, y son esencialmente 4:

- La accion_mover recibe el dicc_entorno, un desplazamiento vertical y horizontal. Primero localiza al capitán ■, después comprueba si hay pared en la dirección pedida y, si el movimiento es válido, calcula la nueva posición. Con esta forma de implementar la función, en algún momento podríamos movernos de forma más libre, es decir, 2 derecha y 1 arriba o 3 abajo y 3 derecha, haciendo más escalable el juego. Hay muchas comprobaciones para mantener coherencia lógica y visual, como marcar tengo_CK si ya estaba antes (para que nos acompañe el coronel), o dejar la casilla vacía una vez se sale de ella.

- La accion_granada tiene una lógica parecida a la acción_mover. Recibe el diccionario, y el lanzamiento vertical y horizontal. Depende de una variable global que impide usarla más de una vez. La acción de granada solo permite lanzar a una casilla adyacente en línea recta y, si en esa casilla hay un soldado, lo elimina del tablero y desactiva el ronquido asociado mediante la función comprobaciones. Se añade el grito en la celda del capitán como señal de que el

evento ocurrió, de esta forma, representamos un evento irreversible en el juego, que cambia por completo la dinámica.

- La accion_salir recibe como input únicamente el dicc_entorno, y comprueba si el capitán está sobre la casilla de salida y si además lleva al coronel. Solo en ese caso se declara True. Si está en salida sin coronel o lleva coronel pero no está en salida, se informa de esto mediante un print. De esta forma nos aseguramos que el jugador llegue a la salida acompañado del Coronel Kurtz, en cualquier otro caso, se devuelve False.

- Por último, la función de impresión de información muestra el estado local de la celda actual, incluyendo percepciones, paredes y progreso del objetivo. Informa al usuario para que pueda tomar decisiones y dirigir la dinámica del juego.

Juego interactivo

Este modo interactivo trata de un bucle (while True) que combina mostrar el estado del juego y pedir una acción al usuario. Basicamente, se muestra la información del entorno, y nosotros según lo que se nos muestra, devolvemos la acción que queremos realizar. Hasta no salir del palacio, o caer en una casilla mortal, no se rompe este bucle.

Al iniciar, se genera un palacio aleatorio y se inicializa la granada y la entrada. En cada iteración se imprime el estado actual y, opcionalmente, se imprime el mapa visual si se quiso así desde el inicio (se debe responder Y en la pregunta de mapa visible). Después se recoge una acción, se ejecuta esta (ya sea movimiento, granada o intentar salir), y se comprueba donde ha caído el capitán, si ha salido o si ha hecho efecto el explosivo. Otorga un control total al usuario.

Juego automático con BFS

El modo automático construye una solución para cada palacio, generado aleatoriamente, según el algoritmo BFS (Breadth First Search). Elegí este algoritmo ya que, como vimos en clase, es completo y como no hay costes de por medios, es óptimo. Es decir, soluciona todas las combinaciones posibles (siempre da solución), y consideraba que quizás no era tan complejo de implementar a nivel código. Para ello primero construimos un conjunto de celdas seguras, estas son todas excepto las celdas mortales. Después, localizamos las posiciones clave de inicio, coronel y salida. La planificación del objetivo depende de si ya tenemos al Coronel Kurtz, o todavía no. Esta diferenciación, permite evitar confusiones o bloqueos de caminos. El algoritmo explora movimientos válidos y seguros (dentro del tablero y a casillas seguras), guarda para cada estado su padre y la acción tomada, y cuando alcanza la salida llevando al coronel reconstruye el plan de acciones.

Una vez obtenido el plan, se ejecuta acción por acción aplicando la misma lógica que usa el modo interactivo, de manera que la ejecución del plan está sujeta a exactamente las mismas reglas del juego. Al finalizar, se intenta salir para formalizar la victoria.

Main

El main de esta primera parte incorpora un menú simple que permite elegir entre modo interactivo y modo automático BFS. También pregunta por si se quiere visualizar el mapa o no. Con esto, mi intención era facilitar la jugabilidad (si se desea), y así comprobar de mejor manera que funcione de forma correcta, aparte de considerarlo una mejora estética.

PARTE 2

Entorno y generación

He planteado la segunda parte de forma muy similar a la primera. La principal diferencia es que se modifica el modelo de celda para permitir que una casilla contenga varios elementos simultáneamente. En el enunciado se nos especificaba que varias trampas podían coincidir, así como la salida, el coronel y el militar. Por tanto, el ejercicio debía de estar preparado para poder “albergar” más de un único elemento en cada celda, y por ello, el atributo celda, para la segunda parte pasa a ser una lista de elementos contenidos en cada casilla.

Esto permite representar casos donde pueden coexistir elementos como salida , coronel CK y militar M, y evita perder información al “pisar” símbolos (sobre todo a nivel visual).

Se mantienen las paredes y estados del juego, y se sustituyen las percepciones por un conjunto adaptado a las trampas, incluyendo indicadores de fuego adyacente, pinchos y dardos adyacentes, además del ronquido del militar.

```
def __init__(self):
    self.celda = [" "]

    self.fuego_ady = False
    self.pinchos_ady = False
    self.dardos_ady = False

    self.ronquido = False
    self.es_salida = False
    self.tengo_CK = False

    self.pared_N = False
    self.pared_S = False
    self.pared_E = False
    self.pared_O = False
    self.grito = False
    self.resplandor = False
```

Definimos dos funciones: añadir y eliminar, para evitar repetir el mismo código varias veces. Con la función añadir, si la celda está vacía, añadimos los elementos que queremos y con la

función de eliminar, quitamos de la lista de contenidos los elementos que queremos. Si esta quedase vacía, se añade el ítem ■.

Al existir una probabilidad de que una celda contuviese más de un elemento, la impresión del palacio debe ser diferente a la de la parte 1. Para ello, definí las casillas como con 3 “capas” (ya que el máximo de elementos juntos son 3), quedando así [| |].

La generación del palacio coloca la entrada en la posición inicial, que de nuevo es (1,1) por definición predeterminada, coloca trampas de fuego, pinchos y dardos en coordenadas aleatorias evitando la entrada, y coloca militar, coronel y salida evitando colisiones con trampas. De nuevo, marca celdas adyacentes con la función comprobaciones adaptada a listas. Marcando cerca de la trampa de fuego, fuego_ady o queroseno, cerca de los pinchos, pinchos_ady o crujidos, cerca de los dardos, dardos_ady o cables, cerca del militar, ronquidos, y por último, cerca de la salida, resplandor.

```

--- INICIO DEL JUEGO ---

ESTADO ACTUAL
-----
Posicion: (1, 1)
Queroseno (fuego adyacente) : False
Crujidos (pinchos adyacentes) : False
Cables (dardos adyacentes) : False
Ronquido (militar adyacentes) : False
Grito : False
-----
PAREDES
Norte: True | Sur: False | Oeste: True | Este: False
-----
OBJETIVO
Es salida : False
Tengo CK : False
-----
Riesgo de celda (1, 2):
- Fuego : 0.0
- Pinchos : 0.0
- Dardos : 0.0
- Militar : 0.0

Riesgo de celda (2, 1):
- Fuego : 0.0
- Pinchos : 0.0
- Dardos : 0.0
- Militar : 0.0

  1      2      3      4      5      6
1 [■] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ]
2 [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ]
3 [ ] [ ] [ ] [M] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ]
4 [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ]
5 [ ] [ ] [ ] [ ] [D] [ ] [ ] [ ] [ ] [ ] [ ] [ ]
6 [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ] [P] [ ] [ ]
Acción: W,A,S,D,G(granada),X(salir)

```

Bayes: Priors, Riesgo, Decisión

Una vez adaptada la estructura al nuevo formato, pasamos a generar funciones auxiliares para la inferencia bayesiana sobre las celdas con trampas.

En esta parte probabilística creamos un diccionario de creencias (dicc_creencias) que mantiene una distribución, uniforme inicialmente, por cada elemento relevante, asignando a cada casilla una probabilidad de contener fuego, pinchos, dardos, militar, salida. Se inicializa con un prior uniforme sobre todas las casillas válidas salvo la entrada, que es 0 ya que se conoce que el inicio no contiene peligros ni el objetivo. De esta forma empezamos desde un

punto de partida neutral que nos permite actualizar creencias solo con información perceptual observada.

La actualización de creencias se aplica de la siguiente manera: si se percibe un estímulo en la celda actual, se conserva probabilidad únicamente en las celdas adyacentes y se anula el resto, y si no se percibe, se anula la probabilidad de la propia celda y adyacentes y se conserva en el resto. Después normalizamos para mantener una distribución válida sobre 1.

El riesgo de una celda se define como la suma de probabilidades de trampas en esa celda más la probabilidad de militar en esa celda. Esto nos permite comparar casillas de forma directa y sirve como criterio para planificar y para tomar decisiones locales cuando no existe un plan, lo que simplifica la estrategia del agente.

Acciones del juego

Las acciones del juego, mantienen la misma estructura que en la parte 1, adaptas a la lógica por listas de la parte 2. De nuevo y de forma resumida:

- La accion_mover comprueba paredes, evalúa Muerte, marca la recogida de CK y mueve al capitán (gestionando contenido con las funciones añadir y eliminar).
- La accion_granada elimina al militar si está en la casilla objetivo y actualiza la percepción de ronquido en el entorno.
- La impresion de información y accion_salir se mantienen igual.

Juego interactivo

Este modo interactivo tampoco difiere mucho del modo interactivo de la parte 1. En cada iteración se actualizan las creencias con los perceptos de la casilla actual, se imprime el estado local, y se muestran además los riesgos para las celdas adyacentes como apoyo a la decisión del usuario.

Luego se ejecuta la acción elegida y se comprueba la muerte. Se mantiene prácticamente idéntica, con la excepción del diccionario de creencias.

Juego automático Bayes

Este modo de juego (posiblemente el más difícil de plantear), intentaba mantener la estructura del BFS de la primera parte, aumentando el conocimiento gracias al `dicc_creencias`. En cada paso actualiza creencias (con la función `actualizar_todas_creencias`, explicada anteriormente), determina el objetivo: ir hacia el coronel si todavía no le hemos

“recogido” y hacia la salida si ya estamos con él, y construye un conjunto de celdas aceptables definido por un umbral de riesgo.

Se intenta calcular un plan con BFS limitado a aquellas celdas aceptables cuyo riesgo sea menor que el p_umbral (predefinido como 0.2), ordenando la expansión por menor riesgo.

Aprovecho este espacio para comentar el impacto que tiene este p_umbral sobre el juego. La medida de 0.2 es bastante acertada, igual un poco estricta. Esto genera que haya situaciones donde no haya un plan bajo el umbral. Por tanto, es crucial.

El mayor problema que he encontrado ha sido el de la evasión de bucles. A veces, la mejor decisión era volver hacia atrás, ya que las celdas adyacentes tenían una probabilidad de riesgo mayor a la del umbral; y de nuevo, ir hacia delante después. De esta forma, entraba en un bucle del que no podía salir. Por tanto, restringí el acceso a las celdas ya visitadas, y creé el modo arriesgado, donde si no existe ningún plan bajo el umbral, el agente elige el vecino válido con menor riesgo, evitando volver inmediatamente a la casilla anterior salvo que sea el objetivo.

Además, si se está en modo arriesgado y todavía se dispone de granada, se decide usarla cuando la probabilidad de militar domina el riesgo comparado con las trampas, porque en ese caso la granada reduce el componente más probable del riesgo. Esta lógica se incluye para que la granada se use de manera racional y no aleatoria.

También sabe que si en la misma celda se encuentran el coronel o la salida, y el militar, debe usar la granada antes de entrar.

Tras decidir un paso, se intenta ejecutar el movimiento. Si el movimiento no produce cambio de posición por bloqueo, se selecciona un vecino alternativo con el siguiente mejor riesgo para no quedar detenido indefinidamente, de nuevo, evitando el bucle.

Finalmente, hasta que el agente entre en una celda que contiene una trampa o un militar (derrota), o llegue a la salida con el coronel (victoria), no se acaba el juego.

Main

El main de esta parte, muy parecido al de la parte 1, nos permite jugar de forma interactiva o ejecutar el modo automático con umbral de riesgo editable. Si se consigue salir del palacio, se lanza automáticamente la fase del río y se resuelve mediante Value Iteration. Existe también la opción de imprimir visualmente el mapa en cada iteración, según se desee.

Proceso de Decisión de Markov (MDP)

Entorno del río

Tras salir con el Coronel Kurtz del palacio, se modela la salida por el río como un proceso de decisión de Markov donde cada estado es una posición en una cuadrícula (según n_filas y $n_columnas$) y las transiciones son estocásticas por efecto de la corriente. Se genera un río con 2 orillas, casillas de río seguras, 2 islas y una salida en la orilla final. Cada columna, excepto las dos orillas, tiene una fuerza de corriente asignada aleatoriamente. Esta representa la probabilidad de ser arrastrado hacia abajo a la hora de realizar un movimiento. Así, no solo “encontramos el camino”, sino que también elegimos la acción óptima.

La función de transición devuelve todas las salidas posibles de ejecutar una acción en un estado, con sus probabilidades y recompensas. En simples palabras: devuelve todos los posibles resultados al realizar una acción, a , desde un estado, s , en forma de tuplas, (nuevo estado, probabilidad, recompensa).

Cuando la acción es bajar, esta se ejecuta con una probabilidad de 1. En cambio, si es otra, hay una probabilidad de ser empujado por la corriente (según la fuerza de la corriente, $river_strength$), hacia abajo. En cualquier caso, si la acción conduce a una casilla inválida (fuera del tablero o isla), el capitán y el coronel se quedan en la misma casilla con una probabilidad de 1.

Las recompensas, según lo indicado en el pdf, son de -1 por cada paso, -100 por chocar con una isla y +100 por llegar a la salida.

	1	2	3	4	5	6	
1	CWCK	R	R	R	R		
2			R	R	R	R	
3			R	R	R	R	
4			R	R	R	R	
5			R	R	R	R	
6	I	R	I	R	R		
7			R	R	R	R	E

Value Iteration

Para resolver el proceso se aplica Value Iteration, calculando una política y un valor óptimo para cada estado, que indican la mejor acción y resultado en cada posición. Inicializamos los valores a cero y se itera aplicando la ecuación de Bellman (sumatorio de los valores

esperados). Se actualiza la política con la acción que maximiza ese retorno esperado y se repite hasta que el cambio máximo en los valores sea insignificante (esto lo hago llevando un control con un valor “controlador”. Cuando este es el mayor de los valores esperados y es muy muy pequeño, significa que las otras políticas apenas mejoran la situación actual, por tanto, rompemos el bucle). De esta forma, obtenemos tanto el valor óptimo como la política óptima.

Simulación del río

Generamos el río, y le aplicamos Value Iteration obteniendo la política óptima: una vez obtenida, se simula una partida siguiendo esa política para demostrar su funcionamiento. En cada paso se elige la acción siguiendo la política, se muestrea el siguiente estado según las probabilidades de la función de transición, se suma la recompensa y se imprime el progreso junto con el tablero. Con esto, podemos visualizar y verificar el funcionamiento.

Kurtz.py

Para finalizar todo el Proyecto, creo un archivo *kurtz.py* que incorpora el contenido de las partes 1 y 2. Este contiene un main que redirige a cualquiera de los dos archivos, dependiendo del usuario.

Unifica la entrega, facilita la evaluación y permite comparar rápidamente ambas partes del Proyecto.

Conclusión

En conclusión, trabajando en este proyecto final no solo he reforzado y aplicado el material estudiado a lo largo de los últimos cuatro meses, sino que también he aprendido a traducirlo a nivel de código, a adaptar algoritmos o incluso a pensar en ellos en una forma distinta.

Durante el proceso ha habido momentos en los que he tenido que apoyarme en fuentes externas para resolver dudas, pero incluso en esas situaciones el resultado ha sido positivo.

Me ha servido para afianzar mejor la teoría y entender con más profundidad el por qué y cómo de cada cosa.

```

--- MENU ---
1: Interactivo
2: Bayes (decisión bajo incertidumbre)
Modo: 2
Mapa visible? (Y/N): n
Umbral p de riesgo (ej 0.2): 0.3
Voy a entrar en (2, 1) con acción S y riesgo=0.5
Usamos granada
No has acertado... Te has quedado sin granada!
Voy a entrar en (2, 1) con acción S y riesgo=0.5
Voy a entrar en (3, 1) con acción S y riesgo=0.0
Voy a entrar en (3, 2) con acción D y riesgo=0.0
Voy a entrar en (3, 3) con acción D y riesgo=0.0
Voy a entrar en (3, 4) con acción D y riesgo=0.0
Voy a entrar en (3, 5) con acción D y riesgo=0.3333333333333333
Has ganado! Has conseguido salir con el Coronel Kurtz.
Ahora te toca salir del rio...
river_strength: {1: 0.0, 2: 0.1, 3: 0.2, 4: 0.8, 5: 0.5, 6: 0.0}
Inicio: (1, 1) Final: (2, 6)

1 2 3 4 5 6
1 |CwCK| |R| |R| |R| |R| |
2 | | |R| |I| |R| |R| |E|
3 | | |R| |R| |R| |R| |
4 | | |R| |R| |R| |R| |
5 | | |I| |R| |R| |R| |
6 | | |R| |R| |R| |R| |
7 | | |R| |R| |R| |R| |

Paso 1: estado = (1, 1) accion = down -> (2, 1) | r=-1 | total = -1

Paso 14: estado = (5, 6) accion = up -> (4, 6) | r=-1 | total = -14

1 2 3 4 5 6
1 | | |R| |R| |R| |R| |
2 | | |R| |I| |R| |R| |E|
3 | | |R| |R| |R| |R| |
4 | | |R| |R| |R| |R| |CwCK|
5 | | |I| |R| |R| |R| |
6 | | |R| |R| |R| |R| |
7 | | |R| |R| |R| |R| |

Paso 15: estado = (4, 6) accion = up -> (3, 6) | r=-1 | total = -15

1 2 3 4 5 6
1 | | |R| |R| |R| |R| |
2 | | |R| |I| |R| |R| |E|
3 | | |R| |R| |R| |R| |CwCK|
4 | | |R| |R| |R| |R| |
5 | | |I| |R| |R| |R| |
6 | | |R| |R| |R| |R| |
7 | | |R| |R| |R| |R| |

Paso 16: estado = (3, 6) accion = up -> (2, 6) | r=100 | total = 85

1 2 3 4 5 6
1 | | |R| |R| |R| |R| |
2 | | |R| |I| |R| |R| |CwCK|
3 | | |R| |R| |R| |R| |
4 | | |R| |R| |R| |R| |
5 | | |I| |R| |R| |R| |
6 | | |R| |R| |R| |R| |
7 | | |R| |R| |R| |R| |

Llegaste a la salida en 15 pasos. Recompensa total: 85

```

1. Ejemplo de simulación completa